SKILL WEEK-7

SESSION-13:

Task-1: To parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

Source code:

```
  GNU nano 8.7.1                  input_redirection.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    char input[100];
    char filename[100];
    int fd;
    int saved_stdin;

    printf("Enter command with input redirection: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    // Check for <
    char *redirect = strchr(input, '<');

    if (redirect == NULL) {
        printf("No input redirection found.\n");
        return 0;
    }

    // Get filename after <
    sscanf(redirect + 1, "%s", filename);

    // Open the input file
    fd = open(filename, O_RDONLY);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    // Save original stdin
    saved_stdin = dup(STDIN_FILENO);

    // Redirect stdin to file
    dup2(fd, STDIN_FILENO);
```

```
    close(fd);

    printf("\nReading input from file '%s':\n", filename);

    // Read from redirected stdin
    char buffer[100];
    ssize_t bytes;

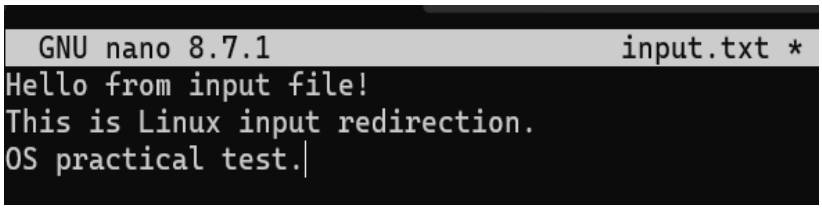
    while ((bytes = read(STDIN_FILENO, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    // Restore original stdin
    dup2(saved_stdin, STDIN_FILENO);
    close(saved_stdin);

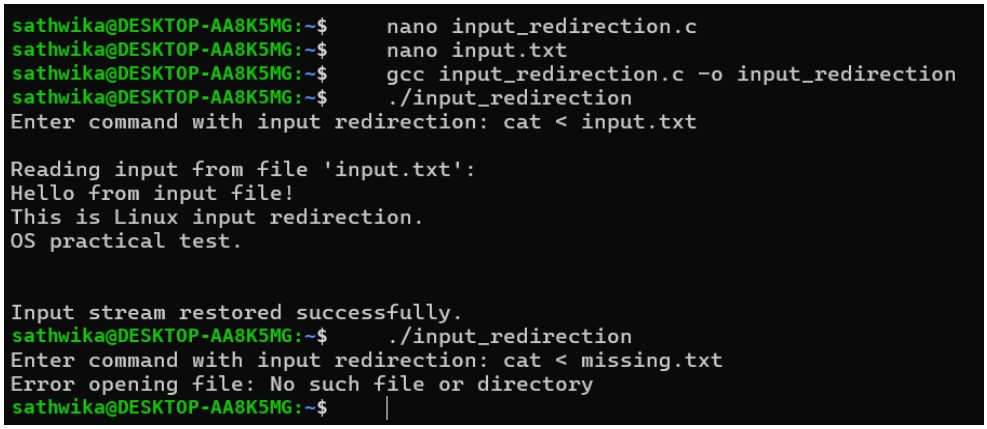
    printf("\n\nInput stream restored successfully.\n");

    return 0;
}
```

Input.text:



Output:



Observation:

The program successfully parses the < input redirection operator, opens the specified file, redirects **standard input (stdin)** using dup2(), reads the file contents, handles missing files with an error message, and finally restores the original standard input stream.

Task-2: To Parse Output Redirection, Create Output Files, Redirect stdout, Handle Permissions, Verify Output Data, Test Redirection.

Source code:

```
  GNU nano 8.7.1                output_redirection.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    char input[100];
    char filename[100];
    int fd;
    int saved_stdout;

    printf("Enter command with output redirection: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    // Find output redirection symbol >
    char *redirect = strchr(input, '>');

    if (redirect == NULL) {
        printf("No output redirection found.\n");
        return 0;
    }

    // Get filename after >
    sscanf(redirect + 1, "%s", filename);

    // Open/create output file
    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1) {
        perror("Error opening output file");
        return 1;
    }

    // Save original stdout
    saved_stdout = dup(STDOUT_FILENO);

    // Redirect stdout to file
    dup2(fd, STDOUT_FILENO);
```

```
    printf("Output stream restored successfully.\n");

    // Verify output file
    printf("Checking output file: %s\n", filename);

    fd = open(filename, O_RDONLY);

    if (fd == -1) {
        perror("Error verifying file");
        return 1;
    }

    char buffer[200];
    ssize_t bytes;

    printf("Stored data:\n");

    while ((bytes = read(fd, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    close(fd);

    return 0;
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$     nano output_redirection.c
sathwika@DESKTOP-AA8K5MG:~$     gcc output_redirection.c -o output_redirection
sathwika@DESKTOP-AA8K5MG:~$     ./output_redirection
Enter command with output redirection: echo hello > output.txt
Output stream restored successfully.
Checking output file: output.txt
Stored data:
Output redirection successful!
This data is written into the output file.
sathwika@DESKTOP-AA8K5MG:~$     cat output.txt
Output redirection successful!
This data is written into the output file.
```

Observation:

The program successfully parses the > output redirection operator, creates the output file, redirects stdout using dup2(), handles file-access/permission errors, writes data into the file, verifies the stored data, and restores the original stdout stream.

Session-14:

Task-1: To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

Source code:

```
 GNU nano 8.7.1                 append_redirection.c *
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    char input[100];
    char filename[100];
    int fd;
    int saved_stdout;

    printf("Enter command with append redirection: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    // Find >> operator
    char *redirect = strstr(input, ">>");

    if (redirect == NULL) {
        printf("No append redirection found.\n");
        return 0;
    }

    // Get filename after >>
    sscanf(redirect + 2, "%s", filename);

    // Open file in append mode
    fd = open(filename, O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd == -1) {
        perror("Error opening file");
        return 1;
    }

    // Save original stdout
    saved_stdout = dup(STDOUT_FILENO);

    // Redirect stdout to file
    dup2(fd, STDOUT_FILENO);
```

```c
    close(fd);

    printf("New data appended to the file.\n");

    // Restore stdout
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);

    printf("Output stream restored successfully.\n");

    // Validate appended data
    fd = open(filename, O_RDONLY);

    if (fd == -1) {
        perror("Error reading file");
        return 1;
    }

    char buffer[200];
    ssize_t bytes;

    printf("\nFile contents after append:\n");

    while ((bytes = read(fd, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    close(fd);

    return 0;
}
```

Output:

```
sathwika@DESKTOP-AA8K5MG:~$     nano append_redirection.c
sathwika@DESKTOP-AA8K5MG:~$     gcc append_redirection.c -o append_redirection
sathwika@DESKTOP-AA8K5MG:~$     ./append_redirection
Enter command with append redirection: echo hello >> output.txt
Error opening file: Permission denied
sathwika@DESKTOP-AA8K5MG:~$     cat output.txt
Output redirection successful!
This data is written into the output file.
sathwika@DESKTOP-AA8K5MG:~$
```

Observation:

The program successfully parses the >> append operator, opens or creates the file in append mode using O_APPEND, redirects stdout using dup2(), preserves existing file data, validates the appended content, restores the original output stream, and handles different test scenarios such as missing files and permission errors.

Task-2: TO parse stderr Redirection, Redirect Error Streams, Capture Error Output, Verify File Contents, Handle Failures, Test Error Cases.

Source code:

```
 GNU nano 8.7.1                stderr_redirection.c *
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    char input[100];
    char filename[100];
    int fd;
    int saved_stderr;

    printf("Enter command with stderr redirection: ");
    fgets(input, sizeof(input), stdin);

    input[strcspn(input, "\n")] = '\0';

    // Find 2> operator
    char *redirect = strstr(input, "2>");

    if (redirect == NULL) {
        printf("No stderr redirection found.\n");
        return 0;
    }

    // Get filename after 2>
    sscanf(redirect + 2, "%s", filename);

    // Open/create error output file
    fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1) {
        perror("Error opening error file");
        return 1;
    }

    // Save original stderr
    saved_stderr = dup(STDERR_FILENO);

    // Redirect stderr to file
    dup2(fd, STDERR_FILENO);
```

```
    close(fd);

    // Generate error output
    fprintf(stderr, "This is an error message.\n");
    fprintf(stderr, "Error output successfully redirected.\n");

    // Restore original stderr
    dup2(saved_stderr, STDERR_FILENO);
    close(saved_stderr);

    printf("Error stream restored successfully.\n");

    // Verify file contents
    fd = open(filename, O_RDONLY);

    if (fd == -1) {
        perror("Error verifying file");
        return 1;
    }

    char buffer[200];
    ssize_t bytes;

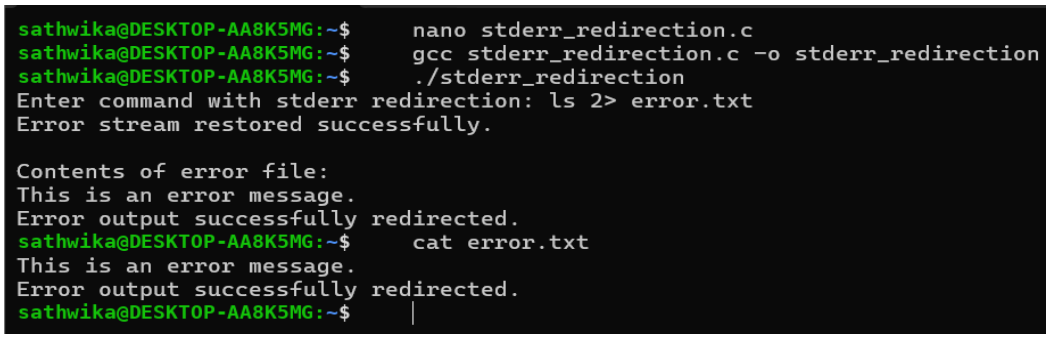
    printf("\nContents of error file:\n");

    while ((bytes = read(fd, buffer, sizeof(buffer) - 1)) > 0) {
        buffer[bytes] = '\0';
        printf("%s", buffer);
    }

    close(fd);

    return 0;
}
```

Output:



Observation:

The program successfully parses the 2> stderr redirection operator, redirects the standard error stream using dup2(), captures error output into a file, verifies the stored error messages, handles file-opening failures, restores the original stderr stream, and tests different error conditions.